

Apostila de Segurança da Informação

Criptografia simétrica, hash, chave pública, IPsec e TLS

Versão em prosa baseada no material simplificado dos slides da disciplina

Contents

Como usar esta apostila	3
1 Cifras de bloco e criptografia simétrica	4
1.1 Cifras de fluxo e cifras de bloco	4
1.2 Confusão e difusão	4
1.3 Rede de Feistel	5
1.4 Parâmetros que afetam a segurança	5
1.5 DES	6
1.6 Efeito avalanche	7
1.7 Por que o DES ficou inseguro	7
1.8 Triple DES	7
1.9 AES	8
1.10 Modos de operação: ECB, CBC, CTR e GCM	8
2 Funções hash, MACs e assinaturas digitais	10
2.1 Função hash	10
2.2 Preenchimento e processamento em blocos	10
2.3 Integridade com hash	10
2.4 O problema da autenticação	10
2.5 MAC e HMAC	10
2.6 Assinaturas digitais	11
2.7 Senhas de mão única	11
2.8 Hash na detecção de intrusão e malware	12
2.9 Pré-imagem, segunda pré-imagem e colisão	12
2.10 Paradoxo do aniversário	12
2.11 SHA e SHA-512	13
3 Criptografia assimétrica, RSA e Diffie-Hellman	14
3.1 Ideia de chave pública	14
3.2 Confidencialidade com chave pública	14
3.3 Autenticação com chave privada	14
3.4 Confidencialidade e autenticação juntas	14
3.5 Funções de mão única com alcapão	15
3.6 RSA	15
3.7 Segurança e tamanho de chave no RSA	15
3.8 Diffie-Hellman	15

3.9	Logaritmo discreto	16
3.10	Ataque man-in-the-middle em Diffie-Hellman	16
4	IPsec	17
4.1	O que é IPsec	17
4.2	IPsec e VPNs	17
4.3	AH e ESP	18
4.4	Associação de Segurança	18
4.5	SAD e SPD	18
4.6	Modo transporte e modo túnel	19
4.7	ESP em modo túnel	20
4.8	IKE	20
5	TLS	21
5.1	Objetivo do TLS	21
5.2	SSL e TLS	21
5.3	Arquitetura do TLS	21
5.4	Conexão TLS e sessão TLS	21
5.5	TLS Record Protocol	22
5.6	Handshake TLS	22
5.7	Change Cipher Spec e Alert	22
5.8	Heartbeat	23
5.9	Heartbleed	23
5.10	TLS versus IPsec	24
6	Comparações essenciais para revisão	25
7	Perguntas de revisão	26
8	Roteiro de revisão de véspera	28

Como usar esta apostila

Esta apostila transforma o conteúdo dos slides em uma explicação contínua, com foco em estudo para prova. A ideia não é reproduzir cada slide, mas organizar o conteúdo em uma sequência mais fácil de entender: primeiro os mecanismos criptográficos básicos, depois as funções de hash e autenticação, em seguida a criptografia assimétrica e, por fim, dois protocolos aplicados: IPsec e TLS.

O melhor uso deste material é ativo. Em vez de apenas ler, tente explicar cada seção em voz alta. Ao final de cada capítulo, use os quadros de revisão como roteiro. Os temas com maior chance de confusão são comparações: hash versus MAC versus assinatura digital; DES versus AES; ECB versus CBC versus GCM; RSA versus Diffie-Hellman; IPsec versus TLS; SAD versus SPD; conexão TLS versus sessão TLS.

Ponto-chave. A disciplina não é apenas sobre decorar nomes de algoritmos. O essencial é saber qual problema cada mecanismo resolve, quais garantias ele oferece e quais limitações permanecem.

Nesta versão, os principais processos foram convertidos em diagramas. Eles servem como atalhos visuais: primeiro observe o fluxo do diagrama e, em seguida, leia a explicação em prosa correspondente.

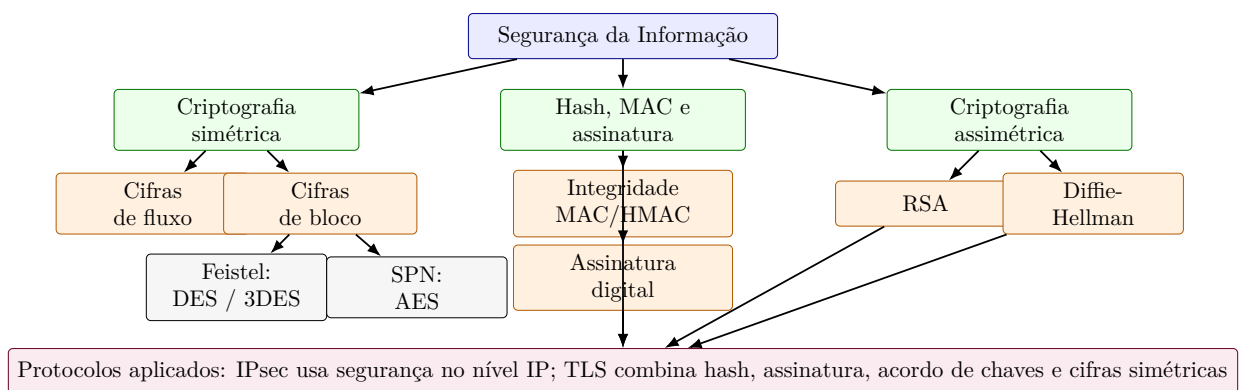


Figure 1: Mapa geral dos temas da apostila: mecanismos criptográficos básicos sustentam protocolos como IPsec e TLS.

Cifras de bloco e criptografia simétrica

1.1 Cifras de fluxo e cifras de bloco

A criptografia simétrica é o cenário em que emissor e receptor compartilham uma mesma chave secreta. Essa chave é usada para transformar texto claro em texto cifrado e, depois, para recuperar o texto claro. Dentro da criptografia simétrica, uma diferença importante é entre cifras de fluxo e cifras de bloco.

Uma **cifra de fluxo** processa a mensagem aos poucos, normalmente bit a bit ou byte a byte. A ideia é produzir um fluxo de chaves, isto é, uma sequência pseudoaleatória controlada por uma chave secreta, e combinar esse fluxo com o texto claro. O exemplo ideal é o one-time pad: se o fluxo de chaves for verdadeiramente aleatório, tiver o mesmo tamanho da mensagem e nunca for reutilizado, ele oferece segurança perfeita. O problema é prático: seria necessário compartilhar previamente uma chave tão longa quanto a mensagem por um canal seguro.

Uma **cifra de bloco** trabalha de outra forma. Ela pega blocos inteiros de dados, por exemplo 64 ou 128 bits, e transforma cada bloco como uma unidade. DES, 3DES e AES são exemplos de cifras de bloco. Para cifrar mensagens maiores que um bloco, usam-se modos de operação, como ECB, CBC, CTR e GCM.

Cifra de fluxo	Cifra de bloco
Processa bit ou byte por vez.	Processa blocos inteiros de tamanho fixo.
Depende de um fluxo de chaves imprevisível.	Depende de uma transformação segura do bloco.
Pode ser muito eficiente em transmissão contínua.	É amplamente usada em arquivos, discos, redes e protocolos.
O one-time pad é o caso ideal.	AES é o padrão moderno mais importante.

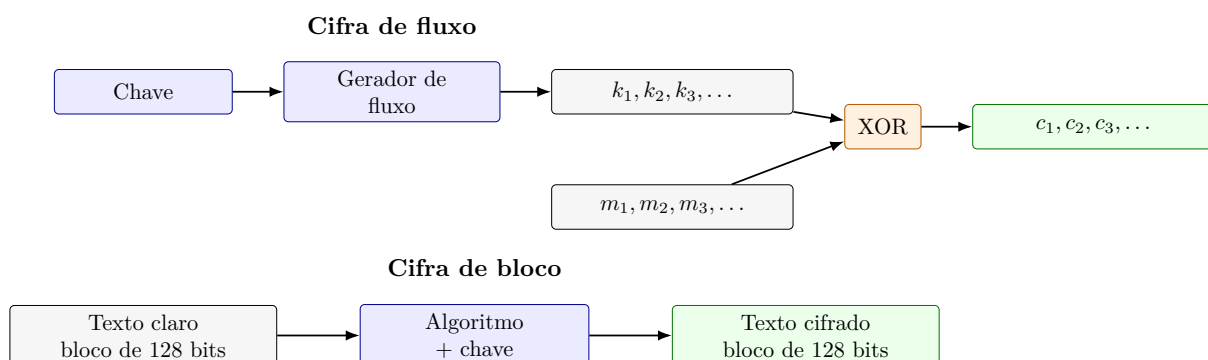


Figure 2: Cifras de fluxo combinam a mensagem com um fluxo de chaves; cifras de bloco transformam blocos inteiros.

1.2 Confusão e difusão

Claude Shannon formulou dois princípios fundamentais para a construção de cifras fortes: **confusão** e **difusão**. Eles aparecem repetidamente em cifras modernas.

A **difusão** busca espalhar a influência dos bits do texto claro por muitos bits do texto cifrado. Se um único bit da mensagem muda, muitos bits do resultado devem mudar. Isso dificulta ataques

baseados em padrões estatísticos, porque a estrutura do texto claro deixa de aparecer claramente no texto cifrado.

A **confusão** busca tornar complexa a relação entre a chave e o texto cifrado. Mesmo que um atacante observe muitos pares de texto claro e texto cifrado, ele não deve conseguir inferir a chave. Na prática, a confusão costuma ser implementada com substituições não lineares, como S-boxes.

Ponto-chave. Difusão esconde padrões do texto claro. Confusão esconde a relação entre chave e texto cifrado. Uma cifra de bloco segura precisa das duas propriedades.

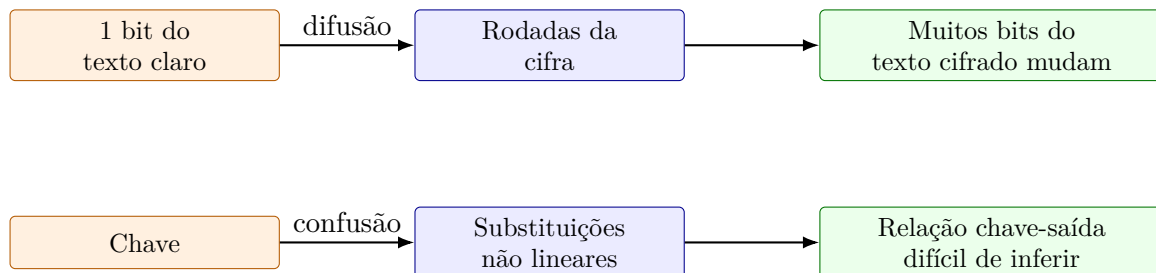


Figure 3: Difusão espalha a influência dos bits da mensagem; confusão dificulta a relação entre chave e resultado.

1.3 Rede de Feistel

A **estrutura de Feistel** é uma forma clássica de construir cifras de bloco. Ela é importante porque permite criar uma transformação reversível mesmo quando a função interna da rodada não é reversível. O bloco de entrada é dividido em duas metades, geralmente chamadas de L_0 e R_0 . Em cada rodada, uma função F é aplicada a uma das metades, junto com uma subchave daquela rodada, e o resultado é combinado com a outra metade usando XOR.

Uma rodada típica de Feistel é descrita por:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Aqui, K_i é a subchave da rodada i . A função F é o núcleo da rodada: ela mistura dados e chave, introduzindo confusão e contribuindo para a difusão ao longo de várias rodadas.

A grande vantagem da rede de Feistel é que a decifração usa a mesma estrutura da encriptação, apenas aplicando as subchaves em ordem inversa. Isso simplifica implementações, especialmente em hardware e software de baixo nível.

1.4 Parâmetros que afetam a segurança

A segurança de uma cifra de Feistel depende de vários parâmetros. O primeiro é o **tamanho do bloco**. Blocos maiores reduzem a repetição de padrões e aumentam o espaço de possíveis entradas, mas podem tornar a implementação mais pesada. Historicamente, 64 bits foi comum, como no DES; hoje, 128 bits é um tamanho muito usado, como no AES.

O segundo parâmetro é o **tamanho da chave**. Chaves curtas são vulneráveis a força bruta. Uma chave de 56 bits, como no DES, já não é segura diante de hardware moderno. Chaves de 128 bits ou mais são consideradas muito mais resistentes para uso prático.

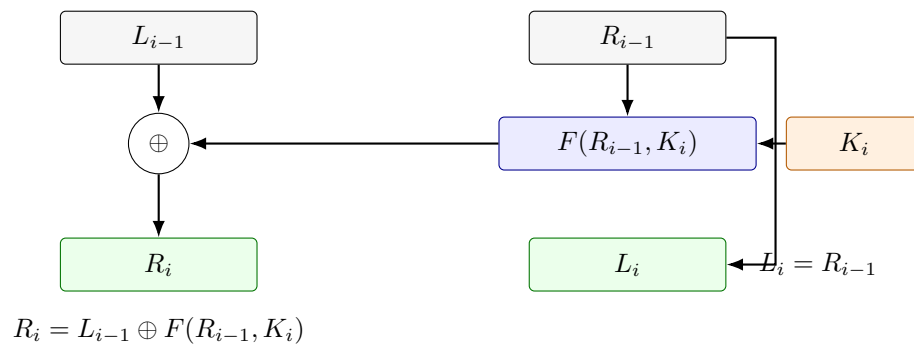


Figure 4: Uma rodada de Feistel: a metade direita alimenta a função F , e a metade esquerda é combinada por XOR.

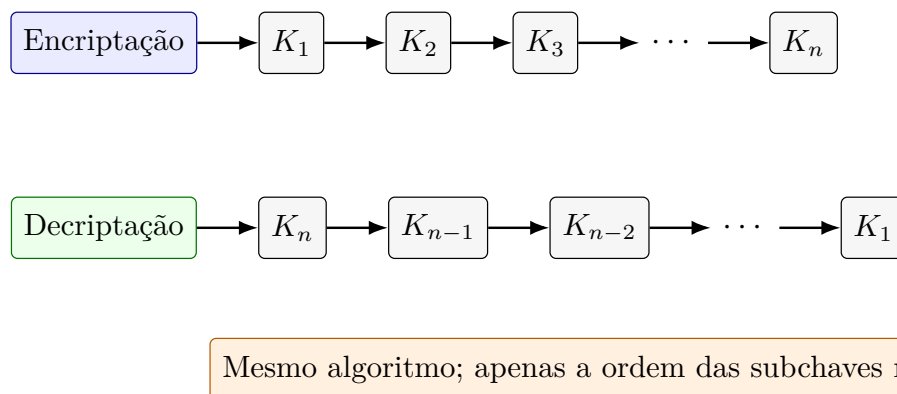


Figure 5: Na rede de Feistel, a deciptação usa as subchaves na ordem inversa.

O terceiro parâmetro é o **número de rodadas**. Uma única rodada não é suficiente, porque a difusão e a confusão ainda são pequenas. Com mais rodadas, cada bit passa a influenciar mais partes do estado interno. O DES usa 16 rodadas, número escolhido para resistir melhor às criptoanálises conhecidas na época.

Também é importante o **escalonamento de chaves**, isto é, o algoritmo que transforma a chave principal em subchaves de rodada. Um escalonamento fraco pode permitir relações previsíveis entre subchaves e facilitar ataques.

1.5 DES

O **Data Encryption Standard**, ou DES, foi durante décadas uma cifra de bloco extremamente importante. Ele trabalha com blocos de 64 bits e usa uma chave efetiva de 56 bits. Embora a chave seja representada em 64 bits, 8 bits são usados para paridade e não contribuem diretamente para a segurança criptográfica.

O DES é baseado em uma rede de Feistel com 16 rodadas. O texto claro passa por uma permutação inicial, é dividido em metades, passa pelas rodadas de Feistel e, ao final, sofre uma permutação final inversa da inicial. A função de rodada do DES expande a metade direita de 32 para 48 bits, combina com uma subchave por XOR, passa por S-boxes e depois por uma permutação.

A função de rodada é importante porque concentra boa parte da confusão do algoritmo. As S-boxes produzem uma transformação não linear, o que dificulta relações simples entre entrada, chave e saída. A repetição por 16 rodadas aumenta a difusão.

Figure 6: Fluxo interno simplificado da função f do DES.

1.6 Efeito avalanche

O **efeito avalanche** é uma propriedade desejável em cifras seguras. Ele significa que uma pequena alteração na entrada, como mudar apenas um bit do texto claro ou da chave, deve causar uma grande alteração no texto cifrado. Idealmente, cerca de metade dos bits da saída deve mudar.

Esse efeito é importante porque impede previsibilidade. Se mudar um bit da mensagem mudasse apenas um bit do texto cifrado, um atacante poderia inferir relações entre entradas e saídas. Em uma cifra bem projetada, a mudança se espalha rapidamente pelas rodadas.

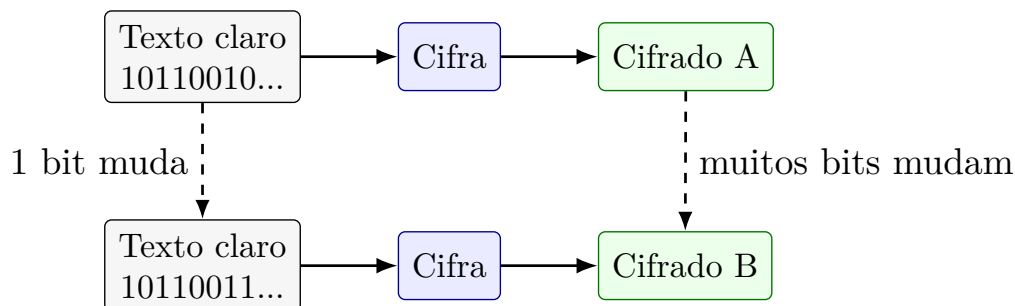


Figure 7: Efeito avalanche: uma mudança pequena na entrada deve gerar uma mudança grande na saída.

1.7 Por que o DES ficou inseguro

O problema principal do DES hoje é o tamanho da chave. Uma chave de 56 bits gera 2^{56} possibilidades, que era um espaço grande no contexto histórico do algoritmo, mas se tornou pequeno diante do avanço de computadores, paralelismo e hardware especializado. A força bruta consiste em testar chaves até encontrar aquela que produz o texto cifrado esperado. Quanto menor a chave, menor o custo desse ataque.

Isso não significa que a estrutura conceitual do DES seja inútil. Pelo contrário: ele é importante para entender redes de Feistel, S-boxes, avalanche e projeto de cifras. Porém, para uso real moderno, recomenda-se AES ou outro algoritmo atual aprovado.

1.8 Triple DES

O **Triple DES**, ou 3DES, surgiu como uma forma de aumentar a segurança usando o próprio DES. Em vez de aplicar o DES uma vez, aplica-se o algoritmo três vezes, em uma sequência conhecida como EDE: encripta, decripta e encripta. A forma geral é:

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

O uso da decriptação no meio não tem a função intuitiva de “desfazer” a cifra; ele foi escolhido para manter compatibilidade com sistemas DES antigos. Se $K_1 = K_2 = K_3$, o 3DES se comporta como DES simples. Com três chaves diferentes, há uma segurança maior; com duas chaves, costuma-se usar $K_1 = K_3$.

O 3DES foi uma solução de transição. Ele aumentava a resistência em comparação ao DES, mas é mais lento e acabou sendo substituído em novos sistemas pelo AES.

1.9 AES

O **Advanced Encryption Standard**, ou AES, é o padrão moderno mais importante de cifra de bloco simétrica. Ele foi selecionado pelo NIST a partir do algoritmo Rijndael. Diferentemente do DES, o AES não é uma rede de Feistel; ele usa uma estrutura de substituição-permutação.

O AES trabalha com blocos de 128 bits e pode usar chaves de 128, 192 ou 256 bits. A diferença entre AES-128, AES-192 e AES-256 está principalmente no tamanho da chave e no número de rodadas. Em termos práticos, AES-128 já é considerado muito forte contra força bruta, e AES-256 oferece uma margem ainda maior.

O processamento interno do AES envolve etapas como substituição de bytes, deslocamento de linhas, mistura de colunas e adição da chave de rodada. Para estudo de prova conceitual, o essencial é entender que o AES substituiu o DES como padrão moderno, trabalha com blocos de 128 bits e oferece tamanhos de chave muito superiores.

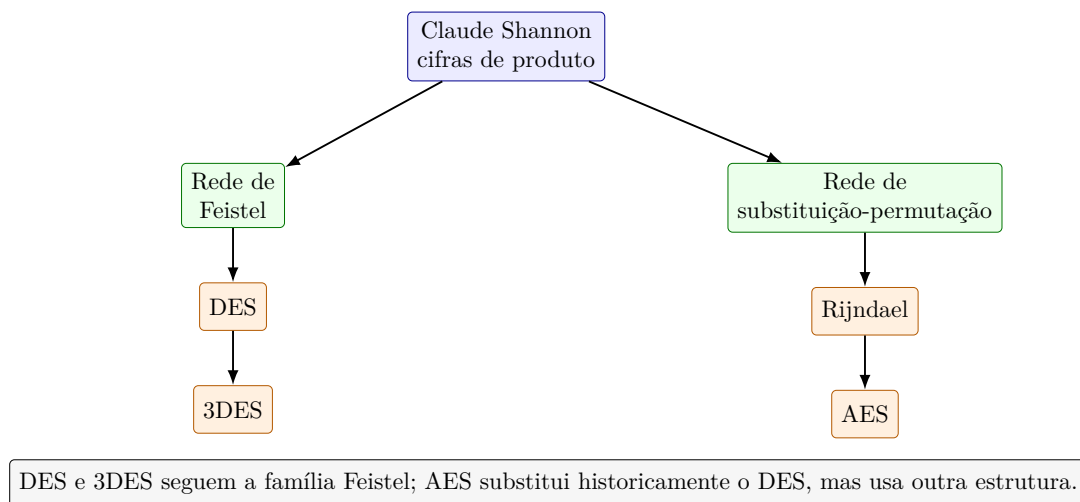


Figure 8: Relação conceitual entre Feistel, DES, 3DES, Rijndael e AES.

1.10 Modos de operação: ECB, CBC, CTR e GCM

Uma cifra de bloco sozinha cifra apenas um bloco. Para mensagens longas, é necessário definir como os blocos serão tratados. Isso é papel dos **modos de operação**.

O **ECB** cifra cada bloco de forma independente. Se dois blocos de texto claro forem iguais, os blocos cifrados correspondentes também serão iguais. Por isso, o ECB revela padrões e não deve ser usado para proteger mensagens estruturadas ou imagens. Ele é simples, mas perigoso.

O **CBC** melhora esse problema ao encadear os blocos. Antes de cifrar um bloco, faz-se XOR com o bloco cifrado anterior. No primeiro bloco, usa-se um vetor de inicialização, o IV. Assim, blocos iguais de texto claro não necessariamente produzem blocos cifrados iguais, desde que o IV seja adequado.

O **CTR** transforma a cifra de bloco em algo parecido com uma cifra de fluxo. Em vez de cifrar diretamente o texto claro, cifra-se uma sequência de contadores. O resultado é combinado com a mensagem por XOR. Isso permite paralelismo e torna o modo eficiente.

O **GCM** combina o modo Counter com autenticação. Ele oferece confidencialidade e também uma tag de autenticação, permitindo detectar alteração dos dados. Por isso, GCM é muito usado em

protocolos modernos, como TLS.

Modo	Ideia	Observação
ECB	Cifra blocos de forma independente.	Vaza padrões.
CBC	Encadeia blocos com XOR e IV.	Melhor que ECB, mas sequencial.
CTR	Cifra contadores e combina por XOR.	Paralelizável.
GCM	CTR mais autenticação.	Confidencialidade e integridade.

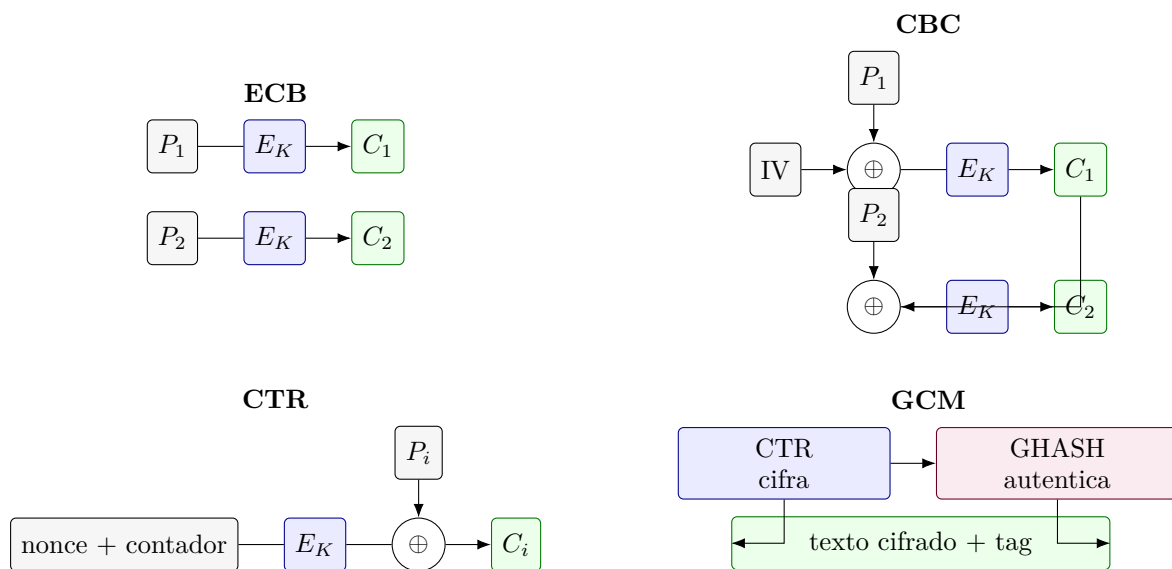


Figure 9: Comparação visual dos modos ECB, CBC, CTR e GCM.

2 Funções hash, MACs e assinaturas digitais

2.1 Função hash

Uma **função hash** recebe uma mensagem de tamanho variável e produz uma saída de tamanho fixo, chamada de hash, resumo ou digest. Escrevemos isso como:

$$h = H(M)$$

A principal utilidade de um hash criptográfico é verificar integridade. Se a mensagem muda, espera-se que o hash mude de forma imprevisível. Assim, ao comparar o hash recebido com o hash recalculado, é possível detectar alterações.

Porém, hash simples não garante autenticidade. Um atacante que consegue modificar a mensagem também pode recalculer o hash e enviar o novo par mensagem-hash. Portanto, para autenticação é necessário usar algum segredo, como em MACs, ou usar assinatura digital.

2.2 Preenchimento e processamento em blocos

Muitas funções hash processam mensagens em blocos de tamanho fixo. Como as mensagens podem ter qualquer tamanho, é necessário aplicar um preenchimento, ou padding. O preenchimento garante que a mensagem final tenha tamanho compatível com o algoritmo e, em muitos casos, inclui também o comprimento original da mensagem.

Esse detalhe é relevante porque mostra que hash não é apenas “reduzir texto”. Existe uma estrutura de processamento que transforma blocos sucessivos em um estado interno, até produzir o digest final.

2.3 Integridade com hash

A verificação de integridade com hash é simples. O emissor calcula $H(M)$ e envia a mensagem junto com o digest. O receptor recalcula $H(M)$ sobre a mensagem recebida e compara com o digest. Se os valores forem iguais, há evidência de que a mensagem não foi alterada acidentalmente. Se forem diferentes, a mensagem foi modificada ou corrompida.

Essa técnica é útil para detectar erros em downloads, arquivos e transmissões. Mas, sozinha, não prova quem gerou a mensagem. Para isso, precisa-se de autenticação.

2.4 O problema da autenticação

Autenticar uma mensagem significa ter garantia de que ela veio de quem deveria vir e não foi alterada no caminho. Um hash simples não basta porque qualquer pessoa pode calcular hash. A solução é proteger o resumo com uma chave ou com uma assinatura.

Existem duas famílias principais de solução. A primeira é o **MAC**, que usa uma chave secreta compartilhada. A segunda é a **assinatura digital**, que usa chave privada e chave pública.

2.5 MAC e HMAC

Um **MAC**, ou Message Authentication Code, é um código de autenticação calculado sobre a mensagem usando uma chave secreta. Apenas quem conhece a chave consegue produzir um MAC válido. Por isso, o receptor, ao recalculer o MAC e obter o mesmo valor, tem evidência de que a mensagem veio de alguém que conhecia a chave e que a mensagem não foi modificada.

Um exemplo comum é o HMAC, que constrói um MAC a partir de uma função hash criptográfica e uma chave secreta. Ele é amplamente usado porque aproveita funções hash bem estudadas, mas adiciona autenticação por chave.

A limitação do MAC é que ele é simétrico. Se Alice e Bob compartilham a mesma chave, ambos conseguem gerar MACs válidos. Isso impede não repúdio forte: Bob não consegue provar para terceiros que foi Alice quem gerou o MAC, porque Bob também tinha a chave.

2.6 Assinaturas digitais

A **assinatura digital** resolve um problema diferente. Ela usa criptografia assimétrica: o emissor assina com sua chave privada, e qualquer pessoa pode verificar usando a chave pública correspondente. Na prática, geralmente não se assina a mensagem inteira diretamente; calcula-se primeiro o hash da mensagem e assina-se o hash.

A assinatura digital oferece integridade, autenticação e não repúdio. Integridade porque qualquer alteração na mensagem muda o hash. Autenticação porque apenas o dono da chave privada deveria conseguir gerar a assinatura. Não repúdio porque terceiros podem verificar a assinatura com a chave pública.

Mecanismo	Usa chave?	Integridade	Autenticidade	Não repúdio
Hash simples	Não	Sim	Não	Não
MAC/HMAC	Sim, secreta	Sim	Sim	Limitado
Assinatura digital	Sim, par as-simétrico	Sim	Sim	Sim

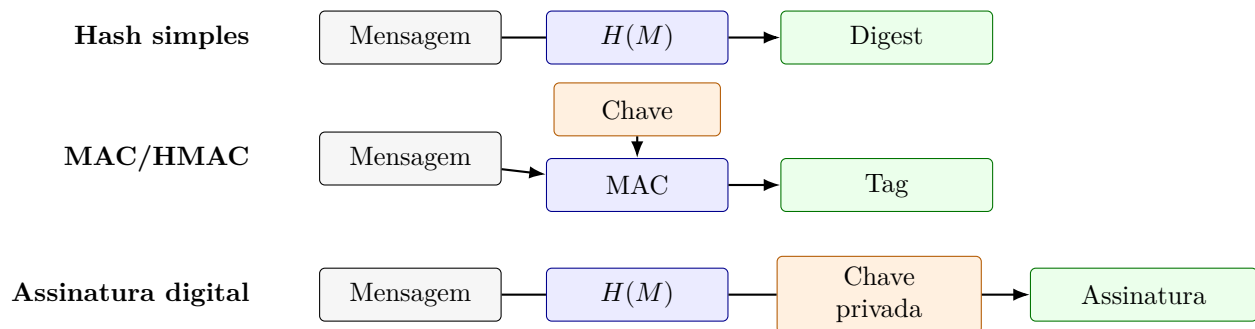


Figure 10: Hash não usa chave; MAC usa chave secreta; assinatura digital usa chave privada e verificação pública.

2.7 Senhas de mão única

Funções hash também são usadas para armazenar senhas. Em vez de guardar a senha em texto claro, o sistema guarda um valor derivado dela. Quando o usuário tenta autenticar, o sistema calcula novamente o valor e compara.

Em sistemas reais, não se deve usar apenas um hash simples da senha. É importante usar sal e algoritmos próprios para senhas, projetados para dificultar ataques de força bruta e tabelas pré-computadas. Ainda assim, o princípio conceitual é o mesmo: a senha não precisa ser recuperada; basta verificar se o valor calculado confere.

2.8 Hash na detecção de intrusão e malware

Outra aplicação é detectar alterações em arquivos. Um sistema pode manter hashes de arquivos importantes e verificar periodicamente se eles mudaram. Se um executável do sistema tem hash diferente do esperado, pode ter sido modificado por um atacante ou por malware.

Antivírus e sistemas de detecção também podem usar hashes para identificar arquivos conhecidos. Essa técnica é útil, mas limitada: pequenas alterações em um malware mudam seu hash. Por isso, detecção moderna não depende apenas de hashes exatos.

2.9 Pré-imagem, segunda pré-imagem e colisão

A segurança de funções hash criptográficas é definida por três resistências principais.

A **resistência à pré-imagem** significa que, dado um digest h , deve ser inviável encontrar uma mensagem x tal que $H(x) = h$. Em outras palavras, o hash deve ser de mão única.

A **resistência à segunda pré-imagem** significa que, dada uma mensagem x , deve ser inviável encontrar outra mensagem $y \neq x$ com o mesmo hash: $H(y) = H(x)$.

A **resistência a colisões** significa que deve ser inviável encontrar qualquer par de mensagens diferentes x e y tais que $H(x) = H(y)$. Como a saída tem tamanho fixo e a entrada pode ter tamanho arbitrário, colisões existem matematicamente. O requisito é que encontrá-las seja computacionalmente inviável.

Propriedade	Pergunta que o atacante tenta resolver
Pré-imagem	Dado h , encontrar x tal que $H(x) = h$.
Segunda pré-imagem	Dado x , encontrar $y \neq x$ com $H(y) = H(x)$.
Colisão	Encontrar qualquer par $x \neq y$ com $H(x) = H(y)$.

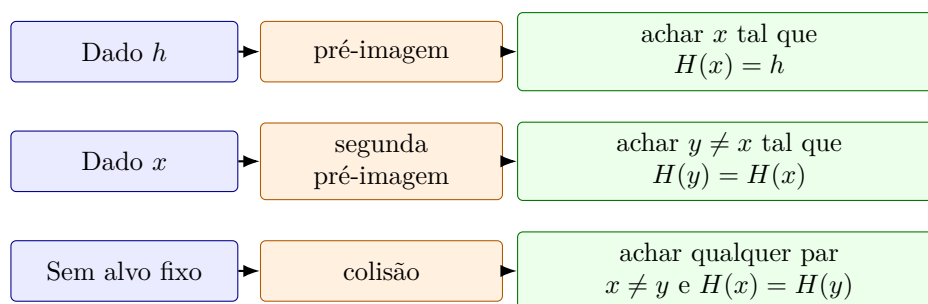


Figure 11: Diferença entre pré-imagem, segunda pré-imagem e colisão.

2.10 Paradoxo do aniversário

O paradoxo do aniversário explica por que ataques de colisão são mais fáceis do que parece. Em uma sala, não é necessário ter 365 pessoas para que duas façam aniversário no mesmo dia; com cerca de 23 pessoas, a chance de colisão já é significativa. O mesmo raciocínio aparece em hashes.

Se uma função hash tem saída de n bits, há 2^n valores possíveis. Para encontrar uma pré-imagem, o esforço esperado é da ordem de 2^n . Para encontrar uma colisão genérica, o esforço cai para aproximadamente $2^{n/2}$. Por isso, um hash de 128 bits oferece apenas cerca de 64 bits de segurança contra colisões, o que é insuficiente para muitos usos modernos.

2.11 SHA e SHA-512

A família SHA é uma das famílias mais conhecidas de funções hash criptográficas. SHA-512 produz digest de 512 bits e processa a mensagem em blocos, mantendo um estado interno que é atualizado por várias rodadas. Para estudo conceitual, o importante é entender que o SHA transforma uma mensagem arbitrária em um resumo fixo e busca garantir resistência a pré-imagem, segunda pré-imagem e colisão.

Não é necessário memorizar todas as constantes e operações internas se a prova for conceitual. O que vale dominar é o papel das funções hash em integridade, autenticação indireta, assinaturas digitais, armazenamento de senhas e detecção de alteração.

3 Criptografia assimétrica, RSA e Diffie-Hellman

3.1 Ideia de chave pública

A criptografia de chave pública introduz uma mudança fundamental: em vez de uma única chave secreta compartilhada, cada usuário possui um par de chaves. Uma é pública e pode ser distribuída; a outra é privada e deve permanecer secreta.

Essa assimetria permite resolver problemas difíceis na criptografia simétrica. Em especial, facilita a distribuição de chaves e permite assinaturas digitais. A segurança depende de funções matemáticas que são fáceis de calcular em uma direção, mas difíceis de inverter sem uma informação secreta.

3.2 Confidencialidade com chave pública

Para enviar uma mensagem confidencial a Bob, Alice cifra a mensagem usando a chave pública de Bob. Como apenas Bob possui a chave privada correspondente, apenas ele deve conseguir decriptar a mensagem. Nesse uso, a chave pública serve para cifrar e a chave privada para decriptar.

O ponto central é que a chave pública pode estar disponível para todos. Não há problema em divulgá-la, desde que a chave privada continue secreta.

3.3 Autenticação com chave privada

Para autenticar uma mensagem, Alice pode usar sua chave privada para assinar. Bob verifica a assinatura usando a chave pública de Alice. Se a verificação for bem-sucedida, Bob tem evidência de que a mensagem foi assinada por quem possuía a chave privada de Alice.

Na prática, assina-se o hash da mensagem, não a mensagem inteira. Isso é mais eficiente e conecta assinatura digital com funções hash.

3.4 Confidencialidade e autenticação juntas

É possível combinar os dois objetivos. Alice pode assinar a mensagem com sua chave privada e depois cifrar o resultado com a chave pública de Bob. Assim, Bob decripta com sua chave privada e verifica a assinatura com a chave pública de Alice.

Essa combinação oferece confidencialidade, porque apenas Bob consegue decriptar, e autenticação, porque apenas Alice deveria conseguir assinar. O custo é maior, pois operações assimétricas são computacionalmente mais pesadas que operações simétricas.

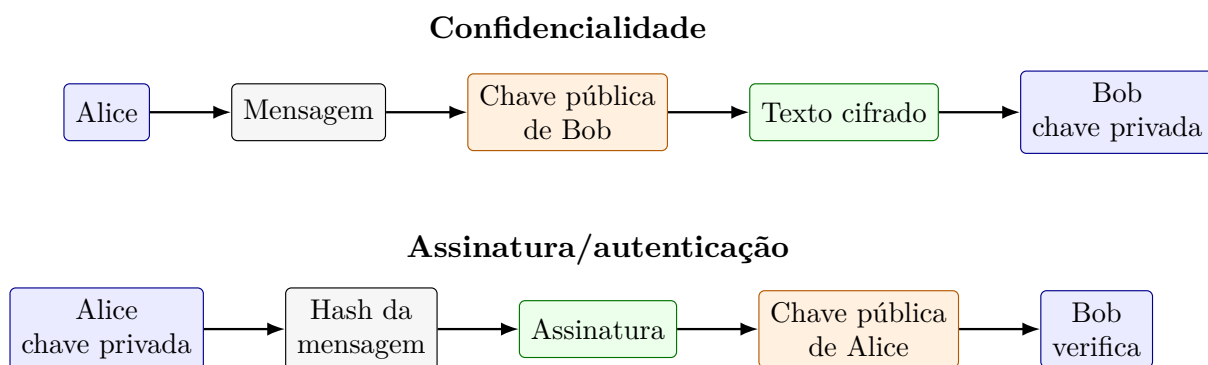


Figure 12: No RSA, a chave pública do destinatário é usada para confidencialidade; a chave privada do emissor é usada para assinatura.

3.5 Funções de mão única com alçapão

Muitos criptossistemas de chave pública se baseiam na ideia de **função de mão única com alçapão**. A função é fácil de calcular em um sentido, mas difícil de inverter. O alçapão é uma informação secreta que torna a inversão viável.

No RSA, por exemplo, é fácil multiplicar dois números primos grandes para obter um módulo. Mas, dado apenas o produto, fatorá-lo de volta nos primos originais é considerado difícil para tamanhos adequados. A chave privada contém informação relacionada a esse alçapão.

3.6 RSA

O **RSA** é um dos criptossistemas de chave pública mais importantes. Ele pode ser usado para cifrar, decifrar, assinar e verificar assinaturas. Seu nome vem de Rivest, Shamir e Adleman.

A geração de chaves RSA segue a ideia geral: escolhem-se dois primos grandes p e q , calcula-se $n = pq$ e, a partir de valores relacionados a p e q , escolhem-se expoentes público e privado. A chave pública inclui n e o expoente público e . A chave privada inclui o expoente privado d .

A operação básica de cifragem pode ser escrita como:

$$C = M^e \bmod n$$

A decifração correspondente é:

$$M = C^d \bmod n$$

Para assinatura, a lógica se inverte: o emissor usa a chave privada para gerar a assinatura, e qualquer verificador usa a chave pública para conferir.

3.7 Segurança e tamanho de chave no RSA

A segurança do RSA depende da dificuldade de fatorar n em seus primos originais p e q . Se um atacante consegue fatorar n , ele pode reconstruir informações necessárias para obter a chave privada.

Como algoritmos e capacidade computacional evoluem, tamanhos de chave precisam ser escolhidos com margem. Chaves pequenas, que poderiam parecer seguras em outro momento histórico, tornam-se inseguras. Em geral, sistemas modernos usam tamanhos de chave RSA bem maiores do que os exemplos didáticos.

Atenção. Exemplos numéricos pequenos de RSA são úteis para entender a matemática, mas não representam segurança real. Em prática, os números precisam ser muito grandes.

3.8 Diffie-Hellman

O protocolo **Diffie-Hellman** tem outro objetivo: permitir que duas partes estabeleçam uma chave secreta comum por meio de um canal público. Ele não é, por si só, uma técnica para cifrar mensagens arbitrárias. Ele é um mecanismo de acordo de chaves.

A ideia básica é que Alice e Bob escolhem segredos privados, calculam valores públicos e trocam esses valores. Em seguida, cada um combina seu segredo privado com o valor público do outro e chega à mesma chave compartilhada.

De forma simplificada, com parâmetros públicos q e α :

$$Y_A = \alpha^{X_A} \bmod q$$

$$Y_B = \alpha^{X_B} \bmod q$$

Depois, Alice calcula:

$$K = Y_B^{X_A} \bmod q$$

Bob calcula:

$$K = Y_A^{X_B} \bmod q$$

Os dois chegam ao mesmo valor porque as potências se equivalem modularmente.

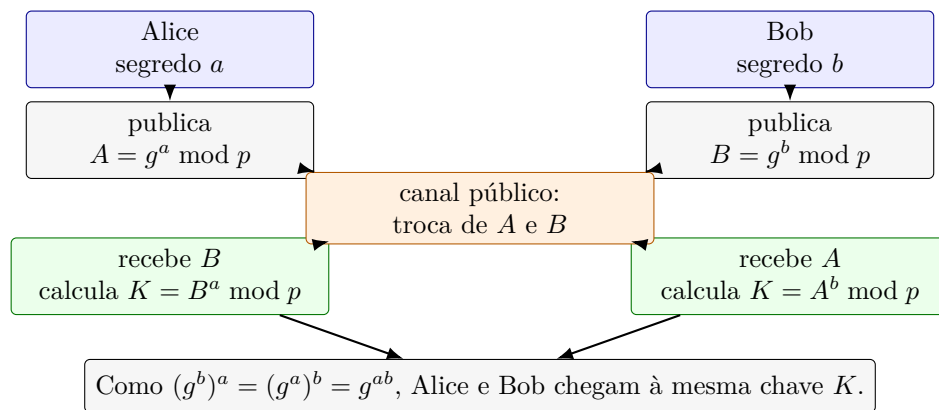


Figure 13: Diffie-Hellman permite obter uma chave compartilhada por um canal público.

3.9 Logaritmo discreto

A segurança do Diffie-Hellman está ligada à dificuldade do logaritmo discreto. É fácil calcular $\alpha^x \bmod q$, mas, dado α , q e $\alpha^x \bmod q$, é difícil recuperar x quando os parâmetros são grandes e bem escolhidos.

Assim, um atacante pode ver os valores públicos trocados, mas não consegue obter facilmente os segredos privados nem a chave final.

3.10 Ataque man-in-the-middle em Diffie-Hellman

O Diffie-Hellman básico tem uma vulnerabilidade importante: ele não autentica os participantes. Um atacante no meio pode interceptar as mensagens de Alice e Bob e estabelecer uma chave com cada um. Alice pensa que compartilha uma chave com Bob, mas compartilha com o atacante. Bob pensa que compartilha uma chave com Alice, mas também compartilha com o atacante.

A solução é autenticar os valores trocados, normalmente com assinaturas digitais e certificados. Em protocolos reais, o acordo de chaves precisa ser combinado com autenticação para evitar esse ataque.

Ponto-chave. RSA e Diffie-Hellman resolvem problemas diferentes. RSA pode cifrar e assinar; Diffie-Hellman estabelece uma chave compartilhada.

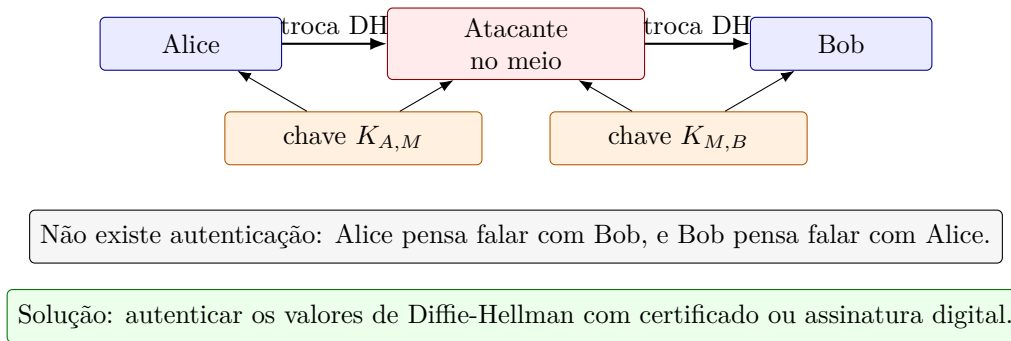


Figure 14: No ataque man-in-the-middle, o Diffie-Hellman sem autenticação cria duas chaves: Alice-atacante e atacante-Bob.

4 IPsec

4.1 O que é IPsec

O **IPsec** é um conjunto de protocolos de segurança para a camada de rede. Ele protege datagramas IP e pode operar entre hosts, roteadores ou entre host e roteador. Uma aplicação importante é a criação de VPNs sobre a Internet pública.

A posição do IPsec na camada de rede é relevante. Diferentemente do TLS, que protege conexões de aplicações sobre TCP, o IPsec protege pacotes IP. Isso significa que ele pode oferecer segurança de forma transparente para várias aplicações, sem que cada aplicação precise implementar criptografia própria.

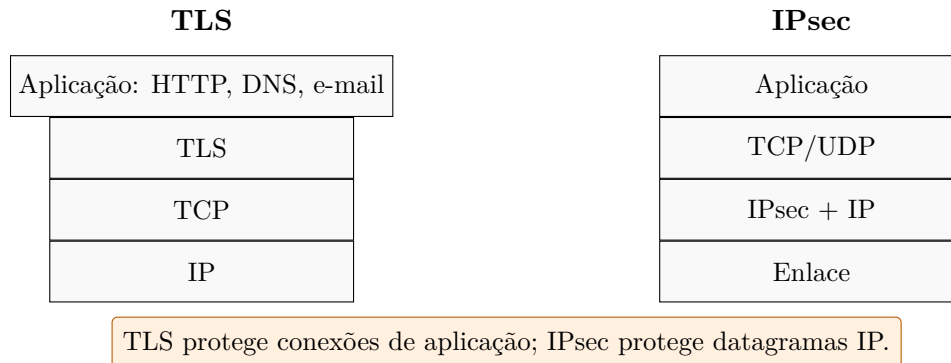


Figure 15: Comparação de posição na pilha: TLS fica acima do TCP; IPsec atua no nível IP.

4.2 IPsec e VPNs

Uma VPN permite que redes privadas se comuniquem com segurança por meio de uma rede pública. Por exemplo, uma matriz e uma filial podem trocar tráfego pela Internet como se estivessem conectadas por uma rede privada. O IPsec protege esse tráfego por criptografia, autenticação e controle de integridade.

Em uma VPN site-to-site, roteadores de borda fazem o trabalho de encapsular, proteger e desencapsular pacotes. Em uma VPN de acesso remoto, notebooks ou dispositivos de usuários podem estabelecer túneis seguros até a rede da organização.

4.3 AH e ESP

O IPsec tem dois protocolos principais de proteção: AH e ESP.

O **AH**, Authentication Header, fornece autenticação e integridade para datagramas IP. Ele permite verificar se o pacote foi alterado e se veio de uma entidade autorizada. Porém, AH não fornece confidencialidade do conteúdo.

O **ESP**, Encapsulating Security Payload, é mais usado na prática porque pode fornecer confidencialidade, além de autenticação e integridade. Com ESP, o conteúdo protegido pode ser criptografado, evitando que terceiros leiam os dados.

Protocolo	Função principal
AH	Autenticação e integridade, sem confidencialidade.
ESP	Confidencialidade e, opcionalmente, autenticação e integridade.

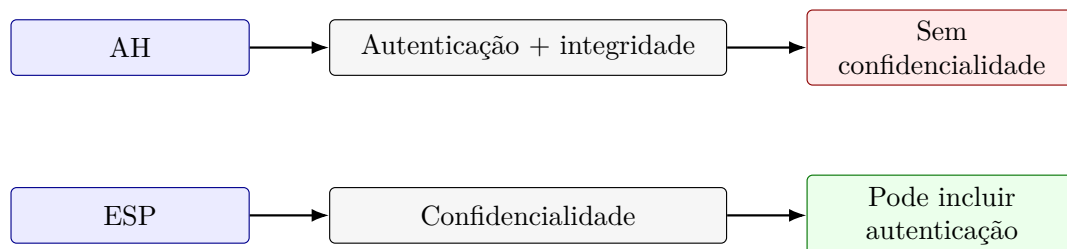


Figure 16: AH autentica e protege integridade; ESP acrescenta proteção de confidencialidade.

4.4 Associação de Segurança

Uma **Associação de Segurança**, ou SA, é uma relação lógica que define como proteger o tráfego IPsec em uma direção. Ela contém parâmetros como algoritmos, chaves, identificador SPI, tempo de vida e informações contra repetição.

Uma característica essencial é que a SA é unidirecional. Para comunicação segura nos dois sentidos, são necessárias duas SAs: uma para cada direção. Em uma VPN com matriz, filial e vários vendedores remotos, o número de SAs cresce conforme o número de fluxos bidirecionais.

4.5 SAD e SPD

Toda implementação IPsec precisa gerenciar muitas SAs. Para isso, usa estruturas de dados importantes: SAD e SPD.

O **SAD**, Security Association Database, armazena os parâmetros das SAs ativas. Ele responde à pergunta: como aplicar a proteção IPsec? Ali ficam chaves, algoritmos, SPI, contadores e demais dados necessários para cifrar, autenticar ou verificar pacotes.

O **SPD**, Security Policy Database, armazena políticas. Ele responde à pergunta: o que fazer com este datagrama? A política pode indicar que o pacote deve ser protegido por IPsec, descartado ou enviado normalmente. O SPD também aponta qual SA deve ser usada quando o pacote precisa de proteção.

Estrutura	Pergunta	Conteúdo
SPD	O que fazer com o datagrama?	Políticas: proteger, descartar ou deixar passar.
SAD	Como aplicar IPsec?	Chaves, algoritmos, SPI, contadores e parâmetros da SA.

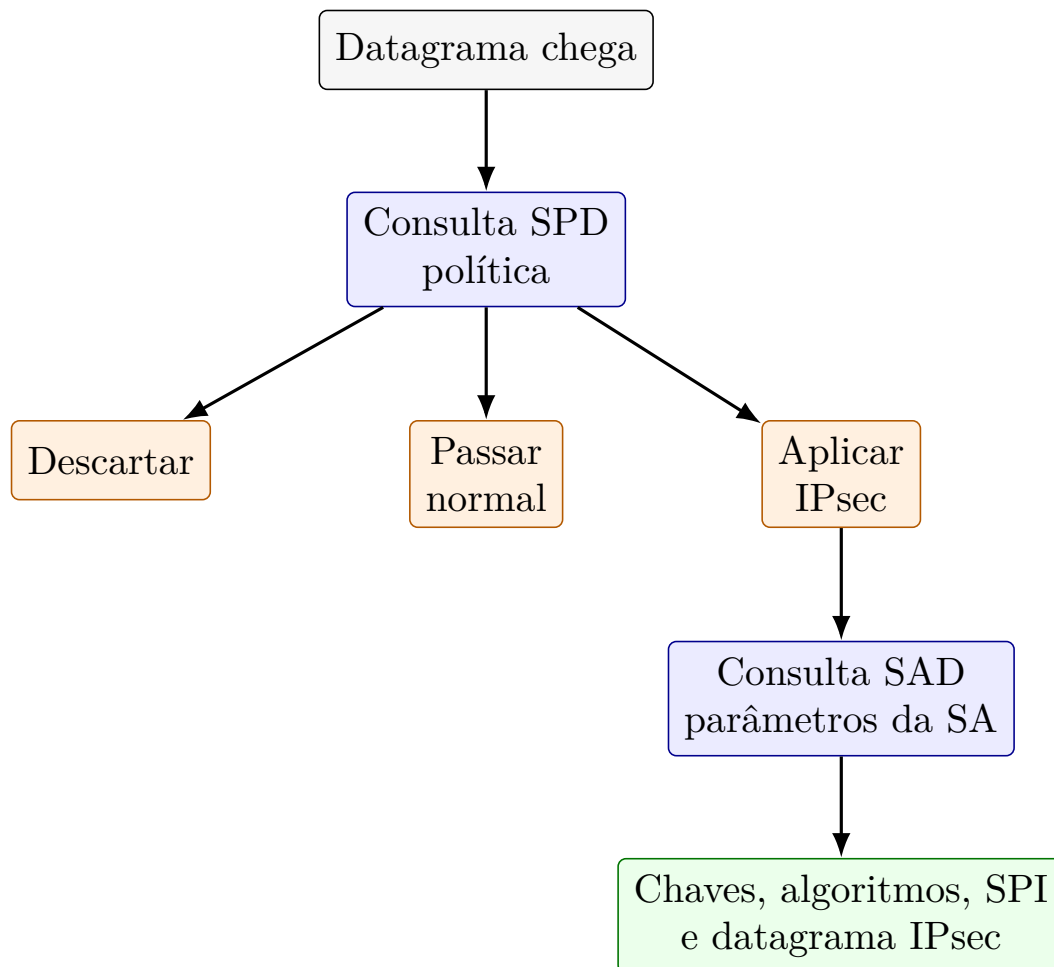


Figure 17: Fluxo de decisão IPsec: o SPD define a política; o SAD fornece os parâmetros da SA.

4.6 Modo transporte e modo túnel

O IPsec pode operar em modo transporte ou modo túnel.

No **modo transporte**, o cabeçalho IP original é mantido e a proteção se aplica principalmente à carga útil do pacote. Esse modo é mais comum em comunicação host-to-host.

No **modo túnel**, o pacote IP original inteiro é encapsulado dentro de um novo pacote IP. O datagrama original vira carga útil protegida. Esse modo é especialmente importante para VPNs, porque permite que roteadores de borda protejam pacotes de redes internas sem que os hosts internos precisem conhecer o IPsec.

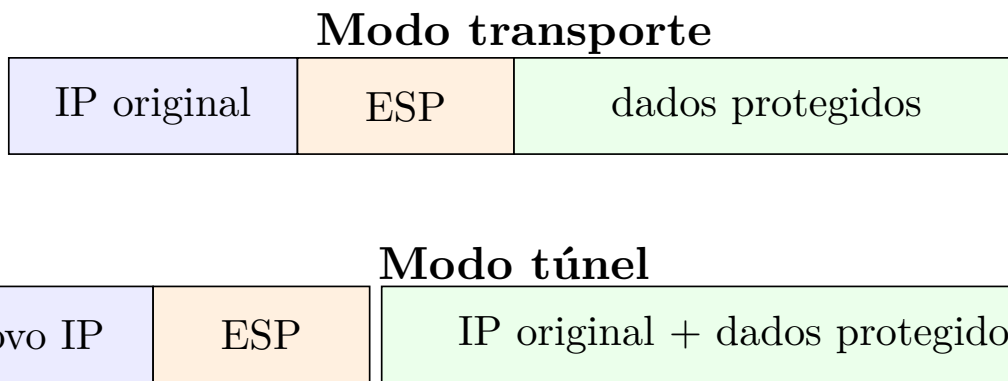


Figure 18: No modo túnel, o pacote IP original inteiro é encapsulado dentro de um novo pacote IP.

4.7 ESP em modo túnel

No ESP em modo túnel, o remetente pega o datagrama IP original, criptografa a parte protegida, adiciona cabeçalhos ESP e cria um novo cabeçalho IP externo. Esse novo pacote atravessa a Internet pública. No destino, o roteador ou host IPsec verifica autenticidade e integridade, decripta o conteúdo e recupera o datagrama original.

Essa estrutura permite esconder o pacote interno e proteger o tráfego entre redes. É por isso que ESP em modo túnel é central em VPNs IPsec.

4.8 IKE

O **IKE**, Internet Key Exchange, é usado para negociar parâmetros e chaves do IPsec. Sem um mecanismo de gerenciamento de chaves, seria necessário configurar manualmente chaves e SAs, o que não escala e aumenta risco operacional.

O IKE ajuda as partes a autenticar uma à outra, escolher algoritmos, estabelecer chaves e criar SAs. Assim, o IPsec pode funcionar de forma mais automatizada e segura.



Figure 19: O IKE automatiza a criação de associações de segurança para o IPsec.

5 TLS

5.1 Objetivo do TLS

O **TLS**, Transport Layer Security, protege comunicações de aplicações sobre TCP. Ele adiciona confidencialidade, integridade e autenticação de ponto final. É o protocolo usado, por exemplo, para proteger conexões HTTPS.

Enquanto o IPsec protege a camada de rede, o TLS atua acima do TCP, entre a aplicação e o transporte. Isso torna o TLS mais próximo das aplicações e mais fácil de implantar em serviços específicos, como sites, APIs e e-mail.

5.2 SSL e TLS

O TLS é sucessor do SSL. Historicamente, SSL veio antes e TLS se consolidou como padronização posterior. Em linguagem cotidiana, muitas pessoas ainda falam “certificado SSL”, mas, tecnicamente, sistemas modernos usam TLS.

O objetivo principal é permitir que cliente e servidor negociem parâmetros criptográficos, autenticuem pelo menos o servidor e estabeleçam chaves simétricas para proteger os dados da conexão.

5.3 Arquitetura do TLS

A arquitetura do TLS pode ser vista em duas camadas. Na parte inferior está o **TLS Record Protocol**, que fragmenta, protege e transmite dados. Acima dele ficam protocolos de controle, como Handshake, Change Cipher Spec e Alert.

O Record Protocol é responsável pela proteção efetiva dos dados. O Handshake negocia os parâmetros e estabelece chaves. O Change Cipher Spec sinaliza a troca para os algoritmos negociados. O Alert comunica erros e encerramentos.

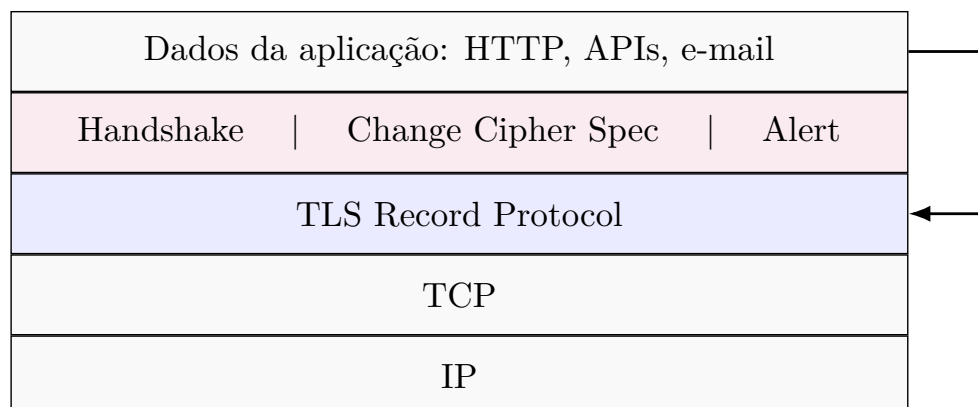


Figure 20: Arquitetura do TLS: o Record Protocol é a base usada para proteger dados e mensagens de controle.

5.4 Conexão TLS e sessão TLS

Uma **conexão TLS** é uma relação transitória entre dois pontos. Ela corresponde ao canal efetivamente usado para enviar dados protegidos.

Uma **sessão TLS** é uma associação mais ampla entre cliente e servidor, criada pelo Handshake. Ela contém parâmetros que podem ser reutilizados para evitar renegociação completa em conexões

futuras. Essa distinção é importante porque o Handshake pode ser caro, principalmente quando envolve criptografia assimétrica.

Conceito	Definição
Conexão TLS	Canal peer-to-peer transitório usado para transmitir dados protegidos.
Sessão TLS	Associação com parâmetros criptográficos que podem ser reutilizados.

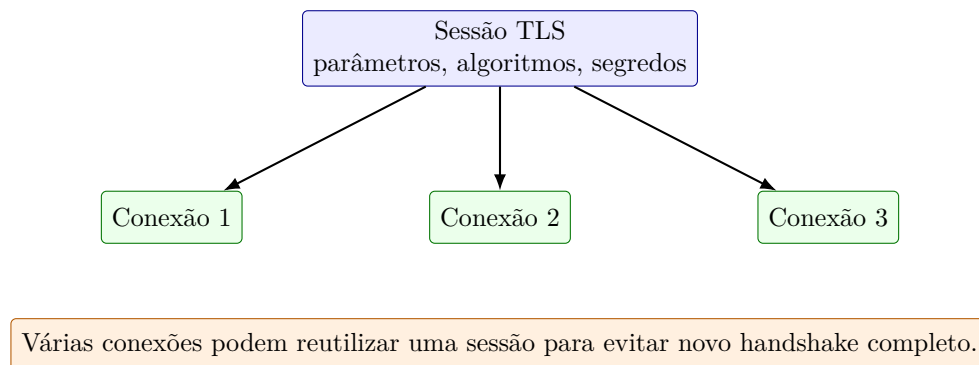


Figure 21: Sessão TLS é uma associação reutilizável; conexão TLS é um canal transitório.

5.5 TLS Record Protocol

O TLS Record Protocol recebe dados das camadas superiores, fragmenta esses dados, pode comprimir, calcula proteção de integridade, cifra e transmite. No receptor, o processo é invertido.

A proteção do Record Protocol depende das chaves e algoritmos escolhidos no Handshake. Depois que a negociação termina, os dados de aplicação são protegidos por criptografia simétrica, que é muito mais eficiente que criptografia assimétrica.

5.6 Handshake TLS

O **Handshake Protocol** negocia a versão do protocolo, escolhe o conjunto criptográfico, autentica o servidor e estabelece segredos compartilhados. Em alguns casos, também autentica o cliente.

De forma geral, o cliente inicia com uma mensagem indicando versões e opções suportadas. O servidor responde escolhendo parâmetros e enviando seu certificado. As partes então executam o mecanismo de troca de chave, derivam chaves simétricas e confirmam que a negociação foi concluída corretamente.

As mensagens finais incluem confirmação de que os algoritmos e chaves negociados passarão a ser usados. A partir desse momento, o Record Protocol transmite dados protegidos.

5.7 Change Cipher Spec e Alert

O **Change Cipher Spec** é uma mensagem simples, mas conceitualmente importante. Ela indica que a parte passará a usar os algoritmos e chaves recém-negociados. Não é a mesma coisa que o Handshake inteiro; é um sinal de ativação do estado criptográfico novo.

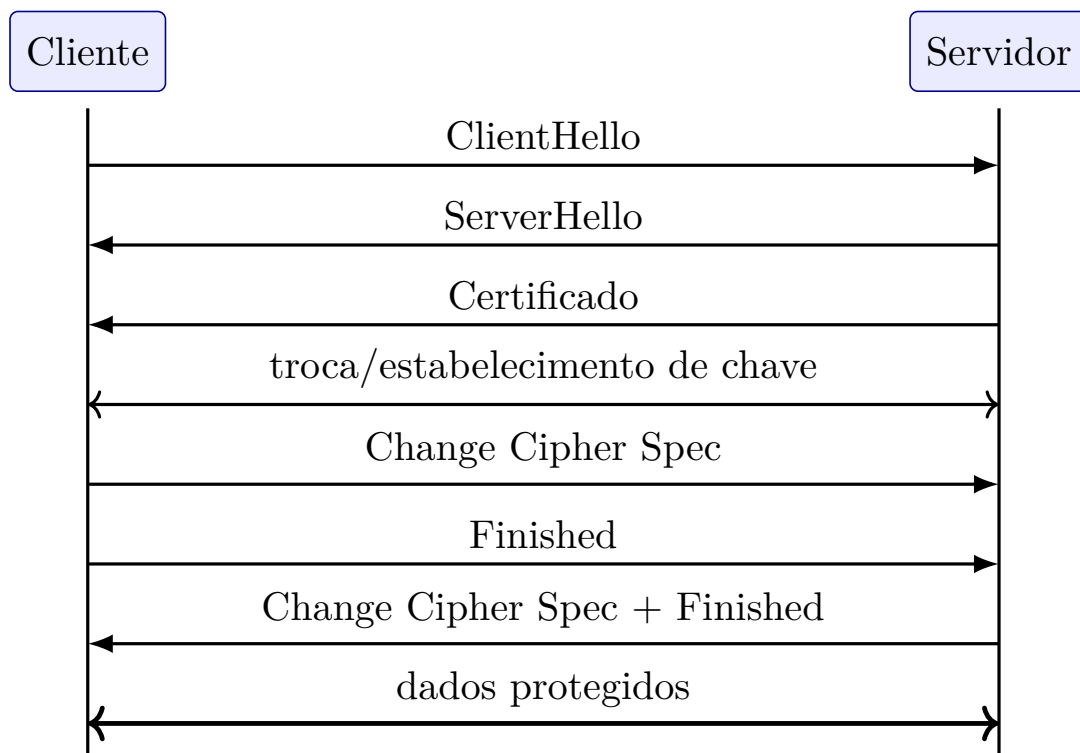


Figure 22: Visão simplificada do Handshake TLS até a ativação das chaves e algoritmos negociados.

O **Alert Protocol** comunica condições de erro, avisos e encerramento. Em protocolos de segurança, encerrar corretamente uma conexão também é importante, pois evita ambiguidades e certos ataques de truncamento.

5.8 Heartbeat

A extensão **Heartbeat** permite verificar se a outra parte ainda está ativa sem renegociar a conexão. Ela funciona como um mecanismo de keep-alive, útil para conexões ociosas que poderiam ser fechadas por firewalls ou intermediários.

A ideia é simples: uma parte envia uma pequena mensagem de batimento, e a outra responde. O problema não está no conceito de keep-alive, mas em implementações incorretas.

5.9 Heartbleed

O **Heartbleed** foi uma vulnerabilidade famosa relacionada à implementação da extensão Heartbeat no OpenSSL. Em termos conceituais, o problema permitia que um atacante solicitasse uma resposta maior do que os dados realmente enviados, fazendo o servidor retornar partes de sua memória.

O impacto foi grave porque essa memória poderia conter dados sensíveis, como cookies de sessão, senhas, informações de usuários e até chaves privadas. Além disso, o ataque era simples de explorar e não deixava necessariamente rastros claros em logs.

Atenção. Heartbleed mostra que usar um protocolo criptográfico forte não basta. A implementação também precisa ser correta, validada e atualizada.

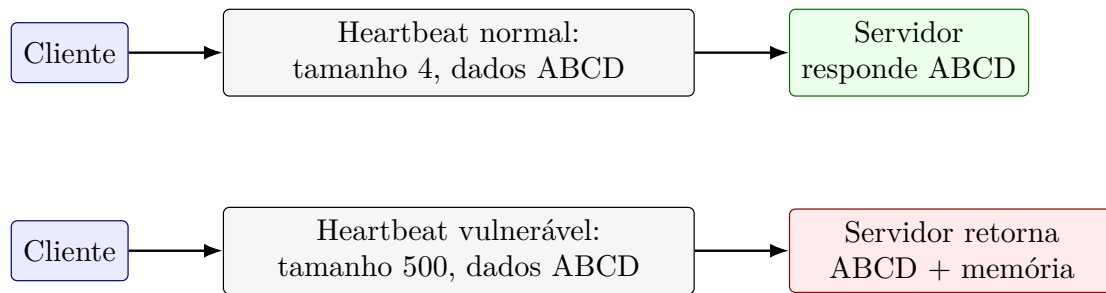


Figure 23: Heartbleed explorava uma inconsistência entre tamanho declarado e dados realmente enviados.

5.10 TLS versus IPsec

TLS e IPsec protegem comunicações, mas em camadas diferentes. O IPsec atua na camada de rede, protegendo pacotes IP e sendo útil para VPNs transparentes. O TLS atua acima do transporte, protegendo conexões de aplicações, como HTTPS.

Na prática, os dois podem coexistir. Uma organização pode usar IPsec para conectar redes e TLS para proteger serviços específicos. A escolha depende do nível da pilha em que se deseja aplicar a segurança e do grau de transparência esperado.

6 Comparações essenciais para revisão

Comparação	Resposta curta
Hash vs MAC	Hash simples não usa chave e não autentica origem. MAC usa chave secreta e autentica a mensagem entre partes que compartilham a chave.
MAC vs assinatura digital	MAC é simétrico e não oferece não repúdio forte. Assinatura digital usa chave privada e pode ser verificada publicamente.
Confusão vs difusão	Confusão dificulta relacionar chave e texto cifrado. Difusão espalha padrões do texto claro pela saída.
DES vs AES	DES usa chave efetiva curta e é obsoleto. AES usa bloco de 128 bits e chaves de 128, 192 ou 256 bits.
DES vs 3DES	3DES aplica DES três vezes para aumentar segurança e manter compatibilidade histórica.
ECB vs CBC	ECB cifra blocos independentemente e revela padrões. CBC encadeia blocos usando XOR com bloco anterior e IV.
CTR vs GCM	CTR gera fluxo a partir de contadores. GCM combina CTR com autenticação por tag.
RSA vs Diffie-Hellman	RSA pode cifrar e assinar. Diffie-Hellman estabelece chave compartilhada.
Chave pública vs privada	A pública pode ser distribuída. A privada deve ficar secreta e permite decryptar ou assinar.
AH vs ESP	AH fornece autenticação e integridade. ESP pode fornecer confidencialidade, integridade e autenticação.
SAD vs SPD	SAD guarda parâmetros das SAs. SPD define políticas sobre o que fazer com pacotes.
Modo transporte vs túnel	Transporte protege principalmente a carga útil. Túnel encapsula o pacote IP original inteiro.
IPsec vs TLS	IPsec protege a camada de rede. TLS protege conexões de aplicações acima do TCP.
Sessão TLS vs conexão TLS	Sessão guarda parâmetros reutilizáveis. Conexão é o canal transitório.
Handshake vs Record Protocol	Handshake negocia parâmetros e chaves. Record Protocol protege os dados transmitidos.

7 Perguntas de revisão

Cifras de bloco

1. Qual é a diferença entre cifra de fluxo e cifra de bloco?
2. O que significam confusão e difusão?
3. Por que a estrutura de Feistel permite usar o mesmo algoritmo para encriptação e deciptação?
4. Quais são os principais parâmetros que influenciam a segurança de uma cifra de bloco?
5. Por que o DES é considerado inseguro atualmente?
6. Qual é a função do efeito avalanche?
7. Por que o modo ECB não é recomendado?
8. O que o GCM adiciona em relação a um modo que fornece apenas confidencialidade?

Hash, MAC e assinatura

1. Hash simples garante autenticidade? Justifique.
2. Qual é a diferença entre MAC e assinatura digital?
3. O que é resistência à pré-imagem?
4. O que é resistência à segunda pré-imagem?
5. O que é resistência a colisões?
6. Por que o paradoxo do aniversário é relevante para hash?
7. Por que geralmente se assina o hash da mensagem, e não a mensagem inteira?

Criptografia assimétrica

1. Como a criptografia de chave pública resolve parte do problema de distribuição de chaves?
2. Como enviar uma mensagem confidencial usando chave pública?
3. Como uma assinatura digital autentica uma mensagem?
4. Em que problema matemático se baseia a segurança do RSA?
5. Qual é o objetivo do Diffie-Hellman?
6. Por que Diffie-Hellman básico é vulnerável a man-in-the-middle?

IPsec e TLS

1. Em qual camada o IPsec atua?
2. Qual é a diferença entre AH e ESP?
3. O que é uma SA e por que ela é unidirecional?
4. Qual é a diferença entre SAD e SPD?
5. Qual é a diferença entre modo transporte e modo túnel?
6. Em qual camada o TLS atua e qual problema ele resolve?

7. Qual é a diferença entre sessão TLS e conexão TLS?
8. Qual é o papel do TLS Record Protocol?
9. O que o Handshake TLS negocia?
10. Por que Heartbleed foi tão grave?

8 Roteiro de revisão de véspera

Na véspera da prova, não tente reler todos os slides. Use uma estratégia de recuperação ativa. Primeiro, cubra a resposta e tente explicar cada comparação da seção anterior. Depois, responda às perguntas de revisão sem consultar o texto. Por fim, volte apenas nos pontos que errou.

A ordem recomendada é:

1. Revisar hash, MAC e assinatura digital.
2. Revisar RSA, Diffie-Hellman e ataque man-in-the-middle.
3. Revisar TLS, especialmente Handshake, Record Protocol e Heartbleed.
4. Revisar IPsec, especialmente ESP, SA, SAD, SPD e modo túnel.
5. Revisar cifras de bloco, principalmente DES, AES e modos de operação.

O conteúdo mais provável de aparecer em perguntas discursivas é aquele que exige comparação: explicar por que hash não autentica, por que ECB vaza padrões, por que Diffie-Hellman precisa de autenticação, por que uma SA é unidirecional, ou como TLS negocia chaves antes de proteger dados de aplicação.

Ponto-chave. Saber explicar as diferenças entre mecanismos vale mais do que decorar tabelas internas de algoritmos.